Software Requirements Specification (SRS)

Etel Ride Platform

Version 1.2 | Etel Group

Document Owner: Etel Family Technology and Business Group (Etel Group)

System: Etel Ride (web app, mobile web app, and related APIs)

Prepared for: INSA Cyber-Security Audit Application, in support of a Road Transport license application with the Ministry of Transport and Logistics

Status: Reverse-engineered from the current production codebase. Etel Ride is delivered from the same repository and backend (delivery/) as Etel Group's parcel/food delivery service (Etel Delivery), since both share one PHP/MySQL backend and one dispatch entrypoint (api/boot.php). This document is scoped strictly to the Ride (passenger/driver) functionality named in the permission request submitted to INSA's Director General; delivery/parcel-order functionality is out of scope and is documented separately. Where a shared backend component (e.g. authentication, the admin panel) also serves the delivery side, this is noted explicitly rather than hidden.

> Version 1.1 adds Section 5 (technology stack, key components, system architecture diagram, data flow diagrams and data flow explanation) requested by the INSA reviewer, and is supplied as PDF. Apart from the v1.2 correction below, Sections 1 to 4 and the appendix are unchanged. **v1.2 correction:** the ride lifecycle now records `arrived` as a real status reached at the drop-off (driver ends the trip), with the passenger's confirmation completing the ride, as verified in RideController.php and described in the TDD/LLD v1.1 Section 3.1.

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of the Etel Ride platform, a ride-hailing system operated by Etel Group. It is prepared to support the INSA (Information Network Security Agency) cyber-security audit application referenced in Etel Group's permission request letter to INSA's Director General, itself a prerequisite for a Road Transport license application with the Ministry of Transport and Logistics.

1.2 Scope

Etel Ride is a ride-hailing service (taxi/bajaj/moto/car/minibus) connecting passengers with onboarded drivers, delivered as installable Progressive Web Apps (Ride for passengers, Driver for drivers), packaged as Android Trusted Web Activities. The system enables:

- Passenger account registration/authentication and ride booking, from fare estimate through completion and rating.
- Driver onboarding (vehicle info, selfie, license, EtelID identity cross-check), ride fulfillment, earnings, and compliance document management.
- Administrative configuration of ride fare rules, driver/ride issue resolution, and driver compliance/vehicle-change approval.

Out of scope for this SRS: Etel Delivery (parcel/food ordering, Courier and Service Giver roles), EtelID (the separate identity-provider system Etel Ride authenticates against for SSO), EtelPay (the separate wallet/payment app used for driver Etel Money top-ups), and other sibling Etel Group systems (EtelGebeya, EtelTaxi, EtelAi).

Note on shared infrastructure: Etel Ride's backend (authentication, the admin panel, database) is not a separate deployment from Etel Delivery's — both run from the same api/boot.php dispatch entrypoint and the same MySQL database, because they are two service lines of one platform. This SRS documents only the requirements, roles, and data directly serving the Ride function; where a shared component (e.g. the bearer-token authentication scheme) also serves Delivery, this is stated rather than omitted, since it cannot accurately be described as Ride-exclusive.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
Driver	The ride-app role that transports passengers, distinct from Etel Delivery's separate Courier role
Ride	A single passenger trip request, tracked in the rides table
Onboarding	The verification process (vehicle info, selfie, license) a driver completes before being allowed to accept rides
TWA	Trusted Web Activity — the Android packaging format wrapping the Ride/Driver PWAs as installable apps
Capability	A named permission string (e.g. pricing.manage) granted to an admin role via the role_permissions table
Equb	A traditional Ethiopian rotating-savings group, offered as an in-app benefit to drivers

1.4 References

- System & Architecture Design Document (SDD) — companion document to this SRS
- Technical Specifications (TDD/LLD) — companion document to this SRS
- Security Documentation — code-level security review of the Ride-relevant surface
- Etel Group's permission request letter to INSA's Director General (submitted separately, referenced as the basis for this audit's scope)

2. Overall Description

2.1 Product Perspective

Etel Ride is one service line of Etel Group's Delivery & Ride platform, sharing its backend and production hosting (cPanel account ethelmfz on ethelmarket.com) with the Etel Delivery service line, and sharing infrastructure more broadly with sibling Etel Group systems (EtelID, EtelPay, EtelTaxi, EtelGebeya, EtelAi). It depends on EtelID as the group-wide identity root: passengers and drivers may register locally or sign in via EtelID SSO (etelid_login), and driver onboarding includes an EtelID identity cross-check. It depends on EtelPay for driver Etel Money top-up verification (server-side transaction verification via etel_pay_verify_tx() before any balance moves).

2.2 Product Functions (Summary)

Full functional detail is in Section 3. At a high level, the product provides:

- Account registration/authentication (local + EtelID SSO) for passengers and drivers
- Ride lifecycle: fare estimate → book → driver match/accept → start → arrive at drop-off (fare billed) → passenger confirms → completed and paid → rating
- Driver operational tooling: online status, earnings, cashout, Etel Money top-up, compliance documents, fuel logs, Equb
- Admin operations relevant to Ride: ride fare-rule and pricing configuration, ride/driver issue resolution, driver compliance and vehicle-change approval, driver account management
- Machine-to-machine integration: EtelAI booking connector and EtelID NFC-tap ride-start action, both authenticated via a shared service key rather than user login

2.3 User Classes and Characteristics

Role	DB representation	Self-registrable?	Summary
Passenger	users.role = 'customer' (Ride passengers are customers, distinguished by app context, not a separate backend role)	Yes (default)	Books and takes rides
Driver	users.role = 'driver' + ride_drivers row (onboarded=1 required)	No — promoted server-side after onboarding	Fulfills ride requests
Admin	users.role = 'admin', scoped by role_permissions capabilities	No — server-side only	Configures ride pricing and resolves ride/driver issues (among other, non-Ride, platform-wide duties — see SDD Section 1 for the shared-admin-panel note)
EtelAI Agent	not a login role — service-key authenticated	N/A	Programmatic ride booking on behalf of an SSO-verified member
EtelID/NFC bridge	not a login role — service-key authenticated	N/A	NFC-tap ride-start action

2.4 Operating Environment

- Server: PHP/MySQL on shared cPanel hosting (PHP 8.1.34)
- Client: Modern mobile/desktop browsers; the Ride and Driver apps are also installable as standalone PWAs / Android TWAs
- Offline behavior: service worker (sw.js) provides cache-first static assets, network-first-with-cache API calls, and an offline fallback page

2.5 Design and Implementation Constraints

- Shared hosting with no SSH access; all deploys go through an in-app file Updater tool
- The Ride and Driver apps ship as versioned, minified, terser-built bundles with a staged canary/stable release-channel system — a source-only fix does not reach production until its bundle is rebuilt and the release channel updated

2.6 Assumptions and Dependencies

- EtelID remains available for SSO and identity verification (selfie/ID cross-check during driver onboarding)
- EtelPay remains available for Etel Money top-up transaction verification

3. Functional Requirements

Requirements are grouped by functional area. Each requirement is traceable to the controller/action(s) implementing it (see the SDD and TDD/LLD for the full API inventory).

3.1 Authentication & Account Management

Note: the authentication mechanism below is shared with Etel Delivery, since both service lines authenticate through the same api/boot.php. It is documented here because Ride cannot function without it, not because it is Ride-exclusive.

ID	Requirement	Source
FR-AUTH-01	The system shall allow a new user to self-register as a passenger (backend role 'customer'); any other role value supplied by the client shall be ignored/forced to customer.	AuthController::register
FR-AUTH-02	The system shall authenticate users via phone + password (login), protected by rate limiting keyed to the phone number.	boot.php login_attempts, AuthController::login
FR-AUTH-03	The system shall support Single Sign-On via EtelID for passengers and drivers (etelid_login) and for admins (admin_etelid_login), the latter never trusting a client-supplied role.	AuthController::etelid_login / admin_etelid_login
FR-AUTH-04	The system shall issue a signed, time-limited bearer token (7-day hard expiry) on successful authentication, and shall reject any token whose embedded version does not match the user's current token_version.	boot.php gen_token/verify_token/get_auth_user
FR-AUTH-05	The system shall allow a user to log out, which immediately invalidates all outstanding tokens for that user across all devices by incrementing token_version.	boot.php logout
FR-AUTH-06	The system shall allow password change with current-password verification, and shall force re-login if the resulting token cannot be safely refreshed.	boot.php change_password

3.2 Ride-Hailing (Passenger / Driver)

ID	Requirement	Source
FR-RIDE-01	A passenger shall be able to request a fare estimate for a given vehicle type before booking, optionally locking the quoted price.	RideController::ride_estimate_fare (quote_token)
FR-RIDE-02	The system shall compute ride fare per vehicle type (bajaj/moto/car/minibus) from base fare, per-km rate, waiting-time rate, and peak-window surge, shared identically between the passenger app and the EtelAI agent integration.	api/lib/ride_fare.php
FR-RIDE-03	A passenger shall be able to book a ride, which transitions through the lifecycle: searching → accepted → in_progress → arrived (trip ended, awaiting passenger confirmation) → completed, or cancelled at applicable stages.	RideController — rides.status, driver_arrived

ID	Requirement	Source
FR-RIDE-04	A driver shall only be permitted to go online, view/accept ride requests, start/complete rides, cash out earnings, or top up the Etel Money wallet after completing onboarding (ride_drivers.onboarded = 1).	RideController::rides_require_onboarded()
FR-RIDE-05	The system shall match nearby available drivers to ride requests using zone/hex-grid based geospatial matching.	api/lib/hexgrid.php
FR-RIDE-06	Both passenger and driver shall be able to rate each other after ride completion, exchange in-ride chat messages, view a live GPS path during the ride, and trigger an SOS alert.	RideController — rate_ride/rate_rider, send_ride_message, ride_path_points, sos_alerts
FR-RIDE-07	A driver shall be able to submit vehicle-detail change requests post-onboarding, which require admin approval before taking effect; the driver's first (onboarding) vehicle declaration is not gated this way.	RideController — driver_vehicle_change_requests + admin_approve/reject_vehicle_change
FR-RIDE-08	A driver shall be able to submit compliance documents (insurance, technical inspection, road fund) with expiry dates, subject to admin approval.	RideController — driver_compliance_requests
FR-RIDE-09	A driver shall be able to cash out earnings and top up an internal Etel Money wallet by submitting an Etel Pay transaction reference, which is server-verified before the top-up enters a pending state, and requires a separate admin approval before the balance is credited.	RideController — driver_cashout, driver_topup_etel_money, admin_approve_etel_topup
FR-RIDE-10	A driver shall be able to join a rotating-savings (Equb) group as an in-app benefit.	EqubController

3.3 Administration (Ride-relevant subset)

The admin panel also configures functionality unrelated to Ride (Etel Delivery order/zone/service-giver management, platform-wide finance/equity tooling). Only the Ride-relevant admin capability is itemized here; see the SDD for the full admin surface and an explicit note on what is out of this audit's scope.

ID	Requirement	Source
FR-ADM-01	The system shall provide role-based, capability-scoped admin access: each admin action shall check both the admin role and a specific granted capability via the role_permissions table.	AdminController — require_admin($pdo, 'capability')
FR-ADM-02	An admin with sufficient capability shall be able to manage ride fare rules and driver accounts.	AdminController, RideController admin actions
FR-ADM-03	An admin with sufficient capability shall be able to resolve ride issues and SOS alerts, and to approve or reject driver compliance documents and vehicle-change requests.	RideController admin actions
FR-ADM-04	The system shall provide a staged release-management system for the Ride and Driver apps, with stable/canary channels, canary tester allowlisting, promotion, and rollback.	app_release_channels / app_canary_testers, api/lib/app_releases.php

3.4 Machine-to-Machine Integrations (Ride-relevant)

ID	Requirement	Source
FR-INT-01	The system shall expose ride quoting and booking actions to the EtelAI agent, authenticated by a shared service key on every call, with member-specific actions additionally resolved to a specific EtelID-SSO-verified local user.	AgentController (agent_ride_quote, agent_book_ride, agent_ride_status, agent_cancel_ride)
FR-INT-02	The system shall expose an NFC-tap ride-start action to EtelID, authenticated by the shared service key, trusting EtelID to have already verified the calling party's role, and shall independently verify the resolved driver's onboarding status before allowing the ride to start.	NfcController::nfc_start_ride

4. Non-Functional Requirements

4.1 Security

- All database access shall use parameterized queries (PDO emulated prepares disabled).
- All user-controllable output rendered into HTML shall be escaped to prevent stored XSS.
- Authentication tokens shall be HMAC-signed, time-limited (7 days), and immediately revocable server-side via a token-version counter.
- File uploads (driver selfie, license, compliance documents) shall be content-sniffed against a MIME whitelist, size-capped, and stored under server-generated filenames to prevent path traversal or executable-content hosting.
- Driver-privileged ride actions (going online, accepting rides, moving money) shall be blocked until onboarding (including EtelID identity cross-check) is complete.

4.2 Availability & Offline Behavior

- The Ride and Driver apps shall function in a degraded offline mode via service worker caching, with a dedicated offline fallback page for navigation requests.

4.3 Performance

- Ride/driver location and status updates shall be delivered to clients via Server-Sent Events (sse_ride_status, sse_driver_stream) rather than polling, to minimize latency and server load.

4.4 Auditability

- Administrative deploys via the Updater tool shall be logged (patch_log) with version, timestamp, and operator.

5. System Overview, Architecture and Data Flow

This section was added in version 1.1 at the INSA reviewer's request. It summarises the technology stack, key components, system architecture and data flows that the requirements in Sections 3 and 4 apply to. Full design detail is in the companion SDD.

5.1 Technology Stack

Layer	Technology	Where it is used
Client	HTML5, CSS3, vanilla JavaScript (no framework); Progressive Web App; installable as Android Trusted Web Activity (TWA)	ride/, driver/, admin.html, login.html; per-app manifest files
Client build	Node.js + terser: per-app source compiled to minified bundles, synced to versioned release folders (stable/canary channels)	js/build-bundles.js, js/releases/{app}/{version}/
Offline / caching	Service worker: cache-first static assets, network-first API calls with 5-minute cache, offline fallback page	sw.js, offline.html
Maps and location	Google Maps JavaScript API (map, markers, polylines, Places Autocomplete), Geocoding API, browser Geolocation	ride and driver apps; server-side geocoding helpers
Server language	PHP 8.1 (procedural, controller-dispatch); no third-party framework	api/boot.php, api/controllers/*
Database	MySQL / MariaDB, utf8mb4, accessed through PDO with prepared statements (emulated prepares disabled); session time zone aligned with PHP (Africa/Addis_Ababa)	api/boot.php, api/migrations/*.sql
Authentication	Custom HMAC-SHA256 signed bearer token with token_version revocation; EtelID SSO; shared service key (hash_equals) for machine-to-machine calls	boot.php, AuthController, etelid_auth.php
Real-time	Server-Sent Events (SSE) for live ride status and driver location	sse_ride_status, sse_driver_stream
Notifications and alerting	Web Push with VAPID (RFC 8292); Telegram Bot API for operational alerts	api/webpush.php, push.php, telegram_webhook.php
Payments integration	EtelPay server-to-server transaction verification (etel_pay_verify_tx)	api/etel_pay_bridge.php
Background jobs	cPanel cron calling key-protected PHP scripts	cron_close_stuck_rides.php, cron_dispatch_scheduled.php, equb_cron.php
Hosting and deployment	Shared cPanel hosting (Apache/LiteSpeed), HTTPS; no SSH; deploys through in-app Admin Updater with backup and patch_log; GitHub Actions workflow for zip-on-merge	api/updater.php, .github/workflows/deploy-zip-on-merge.yml

5.2 Key Components

Component	Type	Responsibility	Trust boundary / security role
Ride app (passenger)	Client PWA/TWA	Registration and login, fare estimate, booking, live tracking, chat, SOS, ratings, favourite and scheduled rides	Untrusted. Holds bearer token only; every action re-authorised server-side.
Driver app	Client PWA/TWA	Onboarding, go online/offline, GPS pings, accept/arrive/start/complete, earnings, cash-out, top-up	Untrusted. Privileged actions gated by rides_require_onboarded().
Admin console	Client web app	Pricing, driver approvals, vehicle-change review, issue and SOS handling, release channels	Untrusted client; each action requires admin role plus capability via role_permissions.
boot.php	Gateway	Request parsing, token / service-key authentication, rate limiting, action whitelist, dispatch	Single choke point for all API traffic.
Controllers	Business logic	Auth, Ride, Admin, Equb, Agent, Nfc, DriverFuel, Push	Ownership checks per ride; capability checks per admin action.
ride_fare.php / hexgrid.php	Library	Fare calculation shared by the app and EtelAI; hex-grid nearest-driver matching	Pure computation; no external input trusted.
SSE endpoints	Real-time	Push ride status and driver position to the participants	Authenticated; scoped to the caller's own ride.
upload.php	Service	Driver selfie, licence and document uploads	Content-sniffed MIME whitelist, size cap, server-generated file names.
Cron scripts	Background	Close stuck rides, dispatch scheduled rides, Equb processing	Standalone, protected by a secret key from .env.
MySQL database	Data store	Users, rides, drivers, GPS path, fares, payouts, permissions	Reached only through PDO prepared statements.
EtelID, EtelPay, EtelAI	External group systems	Identity / SSO, wallet verification, AI booking agent	Shared service key over HTTPS, constant-time comparison.
Google Maps, Telegram, Web Push	External third parties	Maps and geocoding, operational alerts, browser notifications	API key / bot token / VAPID key held server- or domain-restricted.

5.2.1 Component inventory (files)

Component	Responsibility
api/boot.php	Shared dispatch entrypoint: request parsing, auth (token or service key), rate limiting, action routing. Serves both Ride and Delivery.
api/controllers/RideController.php	Ride-hailing lifecycle: fare estimation, booking, driver matching, ride state machine, driver earnings/compliance
api/controllers/AuthController.php	Identity & session (shared with Delivery)

Component	Responsibility
api/controllers/AdminController.php	Platform administration; only its ride fare/pricing, driver-account, and ride/driver-issue-resolution actions are in scope here. Its order/service-giver/zone management actions serve Delivery only and are out of scope for this document.
api/controllers/EqubController.php	Rotating-savings benefit, used by drivers (and, out of scope here, couriers)
api/controllers/AgentController.php	EtelAI machine-to-machine integration; only its ride_* actions are in scope
api/controllers/NfcController.php	EtelID NFC-tap bridge; only nfc_start_ride is in scope (its courier-pickup/delivery actions serve Delivery)
api/lib/ride_fare.php	Ride fare calculation, shared identically between the Ride app and the EtelAI agent integration
api/lib/hexgrid.php	Geospatial hex-grid driver matching
Ride app / Driver app	The two Ride-facing PWAs, each independently versioned and installable as an Android TWA
sw.js (service worker)	Offline caching, install/activate lifecycle, cache versioning (shared mechanism, per-app cache namespace)
EtelID (external)	Group-wide identity provider; SSO login, driver identity cross-check, NFC-tap bridge
EtelPay (external)	Wallet/payment app; verifies driver Etel Money top-up transactions

5.3 System Architecture Diagram

Etel Ride is a four-tier system. Thin PWA/TWA clients talk over HTTPS to a single PHP gateway (boot.php), which authenticates every request and dispatches it to a controller. Controllers read and write the MySQL database and call a small set of external services. Nothing in the client tier is trusted: authorization, ownership and fare calculation are all enforced on the server.

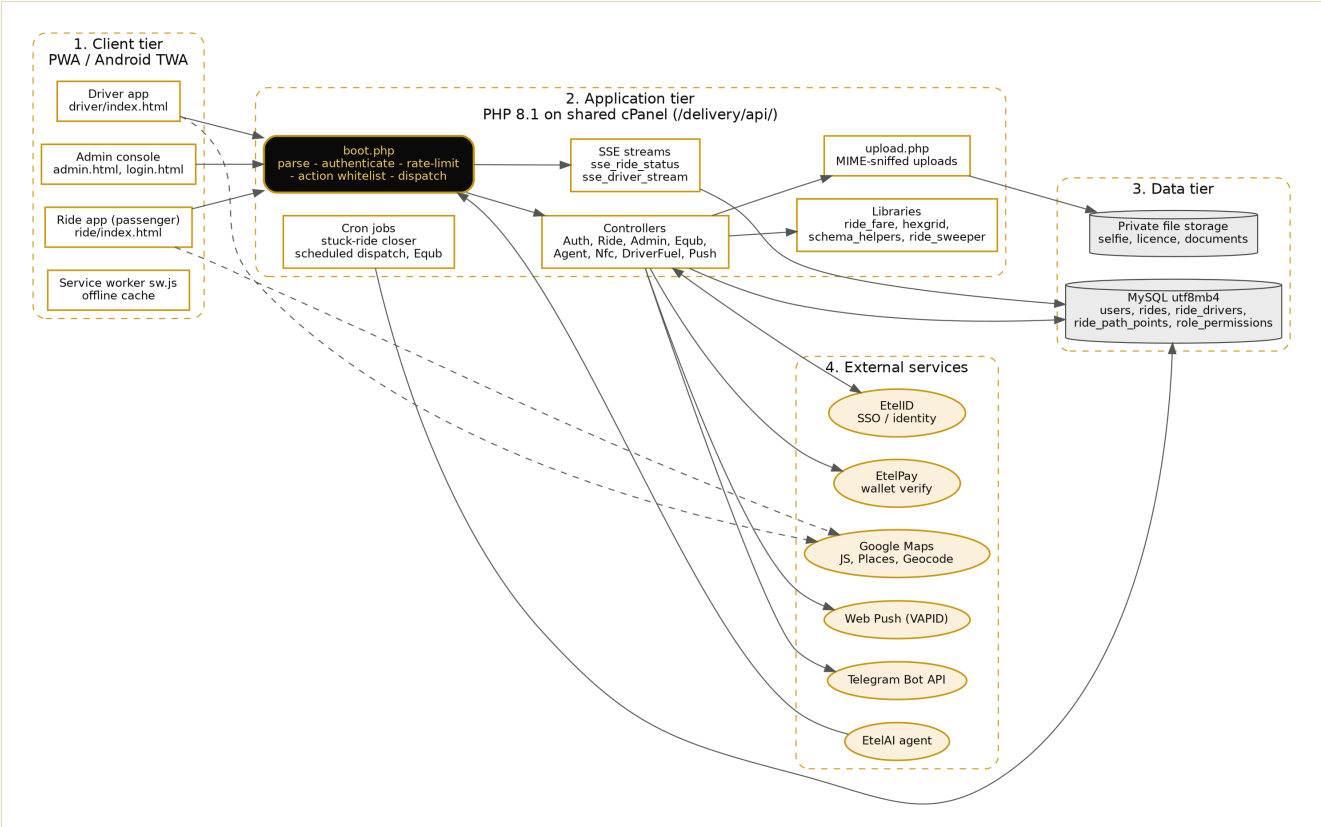


Figure 1 - Etel Ride system architecture (dashed arrows: browser-direct calls to Google Maps)

5.4 Data Flow Diagrams

5.4.1 Level 0 - context diagram

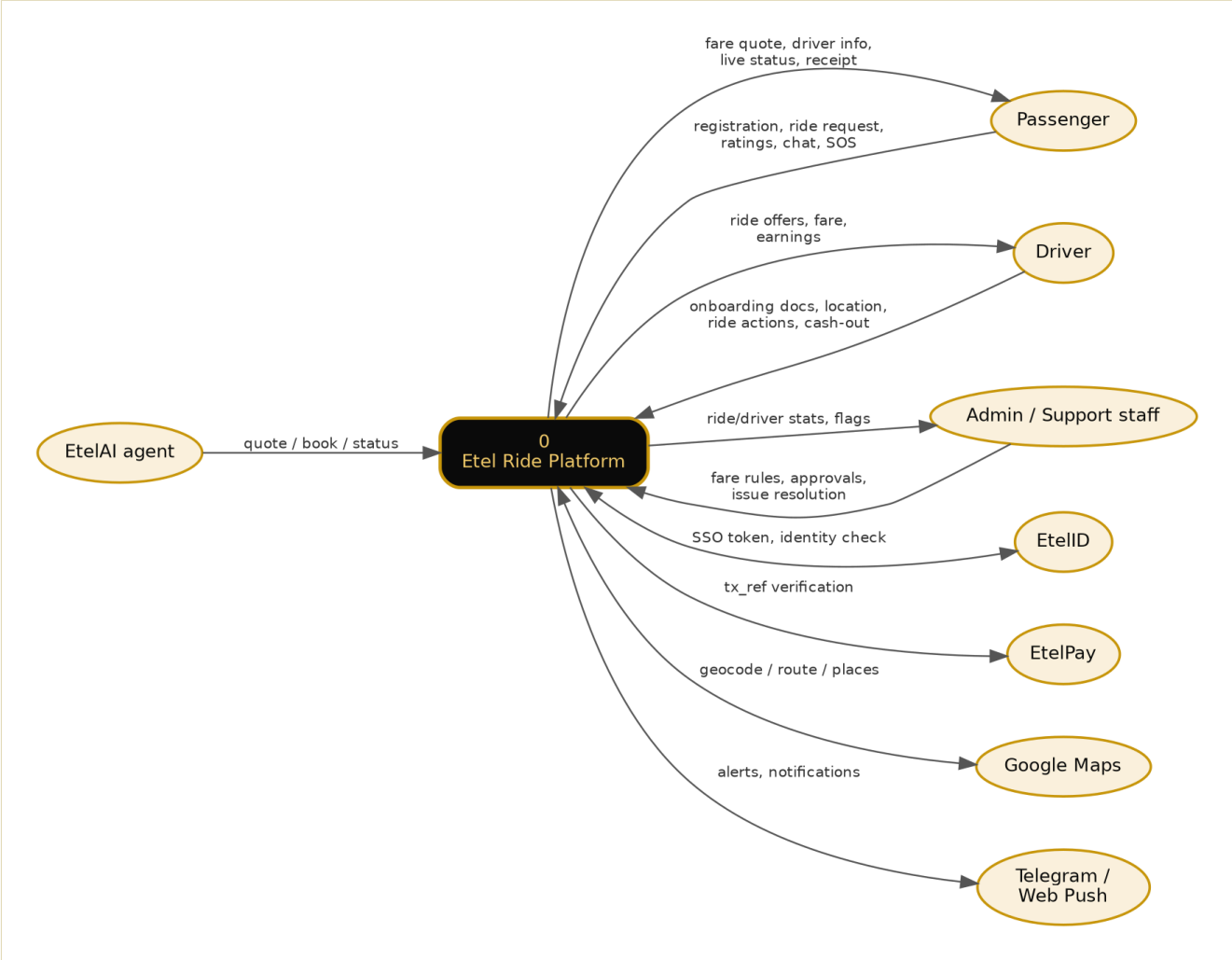


Figure 2 - Level 0 DFD: Etel Ride and its external entities

5.4.2 Level 1 - process decomposition

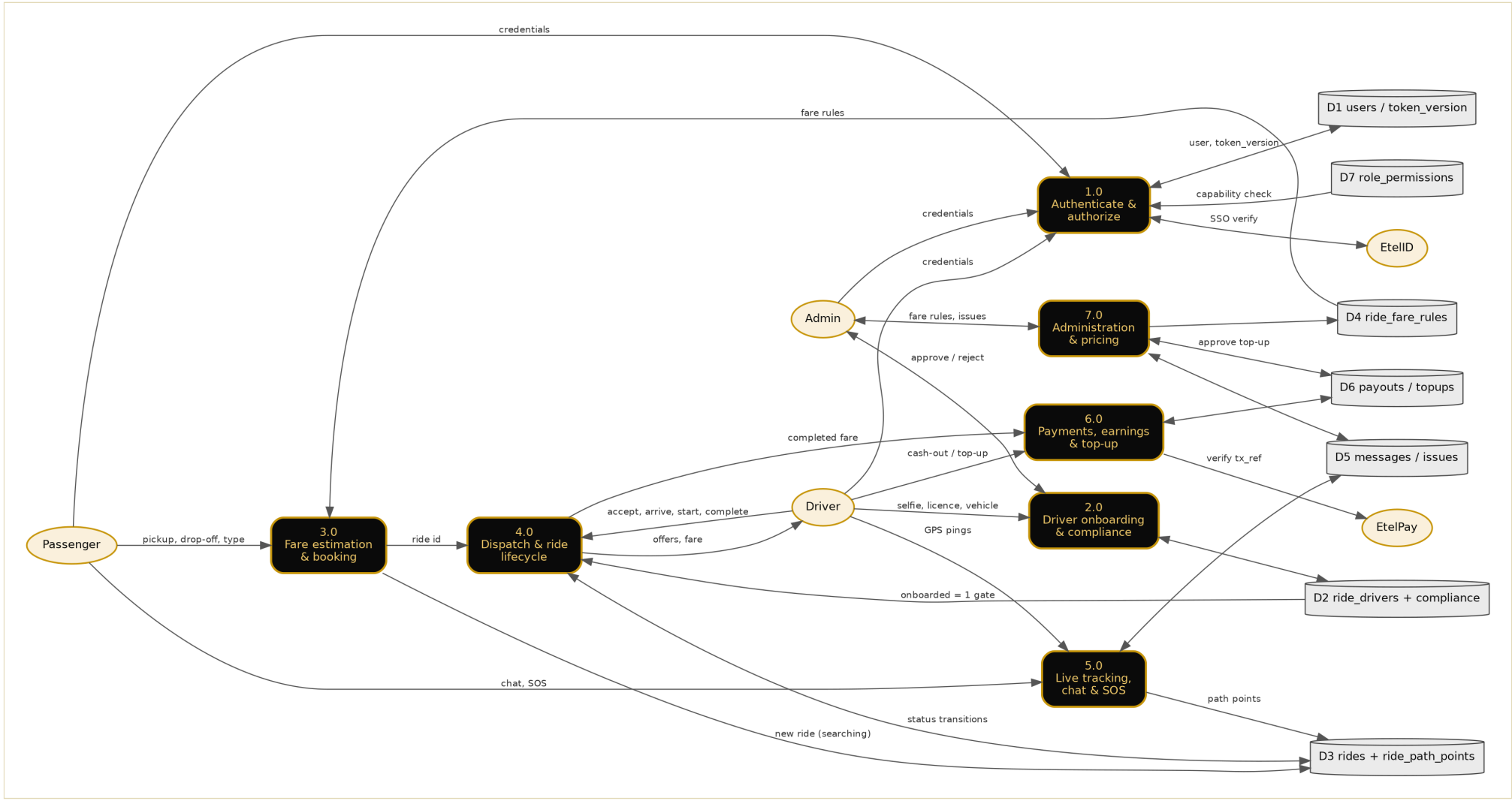


Figure 3 - Level 1 DFD: processes 1.0 to 7.0, data stores D1 to D7, external entities

5.5 Data Flow Explanation

5.5.1 External entities

Entity	Sends to Etel Ride	Receives from Etel Ride
Passenger	Registration, login, pickup and drop-off, vehicle type, chat, SOS, ratings	Fare quote, nearby drivers, driver details, live status and location, receipt
Driver	Onboarding documents (selfie, licence, vehicle), GPS pings, ride actions, cash-out and top-up requests	Ride offers, fare, earnings, announcements
Admin / support staff	Fare rules, driver and vehicle-change decisions, issue resolution, top-up approval	Ride and driver statistics, flagged identities, SOS and issue queues
EtelID	SSO token verification result, role check, NFC-tap ride start	SSO token, driver identity cross-check request
EtelPay	Transaction verification result	tx_ref, expected sender and amount
Google Maps	Map tiles, places, geocode results	Coordinates and address queries
EtelAI agent	Quote, book, status and cancel requests (service key)	Quote, ride status
Telegram / Web Push	-	Operational alerts and rider/driver notifications

5.5.2 Processes

1. **1.0 Authenticate and authorize.** The user submits phone and password (or an EtelID SSO token). Login is rate-limited (login_attempts, keyed by phone). On success the server issues an HMAC-SHA256 bearer token embedding user id, issue time and the user's current token_version. Every later request passes through boot.php, which verifies the signature (hash_equals), the 7-day lifetime and that token_version still matches D1, so logout or password change revokes all tokens at once. Admin actions additionally check D7 (role_permissions) for the required capability.
2. **2.0 Driver onboarding and compliance.** The driver uploads selfie, licence photo and vehicle details through upload.php (content-sniffed MIME whitelist, size cap, server-generated names). The identity is cross-checked with EtelID; mismatches are flagged for admin review. Admin approval sets ride_drivers.onboarded = 1 in D2. Later vehicle changes create a pending request and only take effect when an admin approves.
3. **3.0 Fare estimation and booking.** The passenger sends pickup, drop-off and vehicle type. ride_fare.php reads the per-vehicle rules from D4, computes the fare server-side (client values are never trusted) and returns a quote. On booking, a ride row is written to D3 with status searching.
4. **4.0 Dispatch and ride lifecycle.** hexgrid.php finds nearby online drivers. A driver must pass the onboarding gate (D2) before accepting. The ride moves searching, accepted, in_progress, arrived, completed or cancelled in D3; every transition checks that the caller owns the ride and that the previous status is valid, so a duplicate or replayed action is rejected. Stuck rides are closed by the sweeper and cron script.
5. **5.0 Live tracking, chat and SOS.** The driver app posts GPS points, stored as ride_path_points in D3. SSE endpoints stream status and position to the two ride participants only. Chat messages, issues and SOS events are stored in D5; SOS and operational alerts are forwarded to staff through Telegram and Web Push.
6. **6.0 Payments, earnings and top-up.** When the passenger confirms the trip (or, for cash rides, after a timeout) the fare is posted to the driver's earnings and a 5% commission is deducted from the driver's prepaid Etel Money balance. For an Etel Money top-up the driver supplies a tx_ref, which

is verified server-to-server against EtelPay before a pending row is created in D6; the balance moves only after a second, independent admin approval. Cash-outs are recorded in D6.

7. **7.0 Administration and pricing.** Capability-scoped admin actions edit fare rules (D4), resolve issues (D5), approve drivers and vehicle changes (D2) and approve top-ups (D6). Release channels control which app bundle version users receive.

5.5.3 Typical end-to-end flow (ride request)

1. Passenger app sends ride_estimate_fare to boot.php with its bearer token. boot.php authenticates, whitelists the action and dispatches to RideController.
2. RideController reads D4, computes the quote and returns it. The passenger confirms and book_ride writes a searching ride to D3.
3. Nearby drivers receive the request. A driver calls driver_accept_ride; the onboarding gate and an atomic status update ensure only one driver wins.
4. driver_start_ride moves the ride to in_progress; driver_update_location records the path while SSE keeps the passenger updated.
5. At the drop-off the driver calls driver_arrived (or driver_complete_ride), which stores the fare breakdown and moves the ride to arrived. The passenger confirms with customer_confirm_arrival (wallet payments are verified with EtelPay first); rides_complete_and_pay() then completes the ride, credits wallet-paid earnings, deducts the 5% commission from the driver's Etel Money balance, and the passenger can rate. If the passenger never confirms, the sweeper completes cash rides after a timeout.

5.5.4 Data in transit and at rest

- All client-server and server-server traffic uses HTTPS; bearer tokens travel in the Authorization header only.
- Passwords are stored hashed; tokens are not stored server-side, only the per-user token_version.
- All SQL is parameterised; dynamic identifiers come from fixed whitelists.
- Uploaded documents are stored under server-generated names; runtime deploy backups are blocked from web access (Require all denied).
- Service-to-service calls use a shared key compared with hash_equals(), with a query-parameter fallback for hosts that strip custom headers.

6. Appendix

6.1 Shared-Backend Note

Etel Ride and Etel Delivery are delivered from one shared backend (api/boot.php, one MySQL database) rather than two independently deployed systems. This SRS documents the Ride-facing requirements only; Etel Delivery's parcel/food-order requirements are described in a separate document set. A reviewer auditing only Ride should be aware that the authentication scheme, the admin panel's underlying access-control mechanism, and the production hosting environment are common to both service lines — testing these shared components necessarily also touches infrastructure Delivery relies on, even though this document does not describe Delivery's own functional requirements.

6.2 Open Items Requiring Product/Owner Confirmation

- Resolved in v1.2: 'arrived' is a real rides.status value (the column is widened from ENUM to VARCHAR at runtime), set when the driver ends the trip at the drop-off; the passenger's confirmation then completes the ride. See TDD/LLD v1.1, Section 3.1.
- Android TWA packaging/build configuration was not found within the delivery/ source tree and may live in a separate CI/build repository.